

BTS SIO		Fiche de
BLOC 2	POWERSHELL - INTRODUCTION	synthèse
Admin Sys		N° 2

Ressource du CERTA : Apollonie RAFAELI – Relecture David BALNY

Lien : <https://learn.microsoft.com/fr-fr/powershell/scripting/overview?view=powershell-7.5&viewFallbackFrom=powershell-6>

SOMMAIRE

A. Les commandes Powershell ou « cmdlets ».....	2
A.1 La touche tabulation.....	2
A.2 Les applets de commandes « cmdlet ».....	2
A.3 Méthodes et propriétés associés à une cmdlet.....	2
A.4 Les alias.....	2
A.5 Quelques commandes utiles pour bien travailler.....	2
A.6 Le pipeline.....	3
B. Gestion des dossiers et des fichiers.....	3
B.1 Création, déplacement et suppression.....	4
B.2 Gestion des fichiers.....	4
B.2.1 Lire un fichier.....	4
B.2.2 Écrire dans un fichier.....	5
C. Variables, propriétés et méthodes des objets.....	6
D. Gestion des utilisateurs.....	7
D.1 Le module LocalAccounts.....	7
D.2 Le module ActiveDirectory.....	7
E. Généralités et bonnes pratiques.....	8
E.1 Les commentaires de code.....	8
E.2 Les variables.....	8
E.2.1 Définition.....	8
E.2.2 Types de variables.....	9
E.2.3 Définir une variable.....	10
E.2.4 Afficher les variables.....	10
E.2.5 Imprimer une variable.....	10
E.2.6 Effacer une variable.....	11
E.2.7 Supprimer une variable.....	11
E.2.8 Créer une variable « tableau ».....	11
E.3 Les opérateurs.....	11
E.3.1 Les opérateurs arithmétiques.....	11
E.3.2 Les opérateurs de chaînes.....	12
E.3.3 Les opérateurs logiques.....	12
E.4 Les structures de contrôle IF, ELSEIF, ELSE, SWITCH, TRY, CATCH.....	13
E.5 Les boucles : WHILE, FOR, FOREACH, DO.....	16
E.5.1 Boucle While.....	16
E.5.2 Boucle Do-While.....	16
E.5.3 Boucle For.....	17
E.5.4 Boucle Foreach.....	17

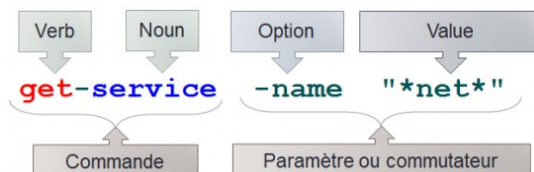
A. LES COMMANDES POWERSHELL OU « CMDLETS »

A.1 LA TOUCHE TABULATION

La touche <tab> permet de compléter automatiquement le nom de la commande, le nom du paramètre, le chemin ou le nom du fichier concerné. L'utilisation répétée de la touche <tab> permet de parcourir tous les choix disponibles.

A.2 LES APPLETS DE COMMANDES « CMDLET »

Dans le monde Powershell, vous remarquerez que les applets de commandes sont composées d'une paire de la forme "verbe"-nom destiné à en faciliter la mémorisation.



Exemple :

```
Start-Service -name eventlog
```

Powershell n'est pas sensible à la casse, les commandes écrites en majuscules ou en minuscules seront interprétées de la même façon. De même, il n'est pas sensible aux espaces ou tabulations.

A.3 MÉTHODES ET PROPRIÉTÉS ASSOCIÉS À UNE CMDLET

Powershell est basé sur des objets, et non du texte. La sortie d'une commande est un objet. À chaque objet est associé des attributs qui définissent son état et des méthodes qui définissent son comportement. Exemples :

- Pour l'objet Voiture vous pourriez visualiser son état par sa couleur : **Voiture.couleur**
- Vous pourriez également connaître son comportement par sa vitesse : **Voiture.vitesse()**

En Powershell, nous pouvons afficher les attributs (ou propriétés) et les méthodes des objets issus des cmdlet. Par exemple, cette commande affiche les propriétés et les méthodes de l'objet **Get-Date**

```
Get-Date | Get-Member
```

A.4 LES ALIAS

Il est possible d'utiliser des *alias* pour raccourcir le nom des *cmdlets*. Ces *alias* ne sont pas forcément clairs toutefois ils permettent de raccourcir des *cmdlets* fréquemment utilisées. La commande suivante permet d'afficher tous les alias pré-définis.

```
Get-Alias
```

Des alias personnalisés peuvent être créés :

```
Set-Alias <nom de l'alias> <commande à exécuter>
```

Exemple :

```
Set-Alias list Get-ChildItem
```



La durée de vie de l'alias est égale à celle de l'invite de commande Powershell. Si on la ferme, l'alias personnalisé est perdu.

A.5 QUELQUES COMMANDES UTILES POUR BIEN TRAVAILLER

Afficher toutes les commandes disponibles

```
Get-Command
```

On peut ajouter des arguments pour chercher suivant un critère particulier

```
Get-Command -verb "<nom du verbe>"  
Get-Command -noun "<nom à chercher>"
```

Afficher de l'aide

```
Get-Help <nom de la commande>
```

Pour afficher l'aide concernant une commande avec des exemples

```
Get-Help <nom de la commande> -examples
```

Il est possible aussi d'utiliser

```
<nom de la commande> -?
```

Par exemple :

```
Get-Command -?
```

Effacer le contenu d'une console

```
Clear-Host
```

Afficher l'historique de la console

```
Get-History
```

Avoir la liste des processus en cours d'exécution sur la machine

```
Get-Process
```

A.6 LE PIPELINE

L'opérateur de pipeline (| : barre verticale) permet de canaliser la commande, c'est-à-dire de passer la sortie d'une commande à une autre commande, en tant qu'entrée.

Exemple, le résultat de la commande Get-Date est mis en forme par la commande Format-Table :

```
Get-Date | Format-Table
```

Date	Day	DayOfWeek	DayOfYear	Hour	Kind	Millisecond	Minute
26/02/26...	26	Friday	57	20	Local	875	54

Exemple, le résultat de la commande ipconfig est transmis à la commande findstr qui extrait les lignes comportant le mot « Adresse » :

```
ipconfig | findstr "Adresse"
```

```
Adresse IP. ....?.. : 192.168.10.100  
Adresse IP. ....?.. : 10.0.0.1
```

B. GESTION DES DOSSIERS ET DES FICHIERS

B.1 CRÉATION, DÉPLACEMENT ET SUPPRESSION

Voici une série de commande permettant de gérer les fichiers et dossiers avec Powershell.

Afficher le chemin du dossier courant

```
Get-Location
```

Afficher le contenu d'un dossier

```
Get-ChildItem  
Get-ChildItem "c:\temp"
```

Se déplacer dans les dossiers

```
Set-Location "c:\temp"
```

Créer un fichier

```
New-Item -Name "fichier.txt" -ItemType file -Value "Test"
```

Créer un dossier

```
New-Item -Name "dossier" -ItemType directory
```

Supprimer un fichier ou un dossier

```
Remove-Item "c:\temp\*.txt"
```

Déplacer des fichiers

```
Move-Item -Path *.txt -Destination "c:\temp\"
```

Déplacer un dossier

```
Move-Item -Path .\dossier -Destination .\temp\dossier  
Move-Item -Path .\temp\dossier -Destination .\temp\dossier2"
```

Renommer un fichier ou un dossier

```
Rename-Item -Path fichier.txt -NewName fichier2.txt  
Rename-Item -Path dossier -NewName dossier2
```

Copier un fichier ou un dossier

```
Copy-Item -Path .\fichier.txt -Destination c:\temp  
Copy-Item -Path dossier -Destination c:\temp -Recurse
```

B.2 GESTION DES FICHIERS

B.2.1 Lire un fichier

La commande Get-Content permet de lire en une seule étape les données du fichier.

```
Get-Content -Path c:\temp\test.txt  
Regnier/Daniel/Laon  
Loisel/Pierre/Amiens
```

Get-Content traite les données lues du fichier comme un tableau, avec une ligne par élément du tableau. Exemple, lecture de la deuxième ligne du fichier (indice 1 du tableau \$ligne qui commence à 0) :

```
$ligne=Get-Content -Path c:\temp\test.txt; $ligne[1]  
Loisel/Pierre/Amiens
```

La même chose sans la variable \$ligne :

```
(Get-Content C:\temp\test.txt)[1]
```

Parcours des lignes du fichier comme une collection avec ForEach.

```
ForEach($ligne in Get-Content c:\temp\test.txt) {$ligne}  
Regnier/Daniel/Laon  
Loisel/Pierre/Amiens
```

Exemple, parcourir le fichier pour afficher une information contenue dans la ligne :

```
ForEach($ligne in Get-Content c:\temp\test.txt) {$mot=$ligne.split("/");  
$mot[0]}
```

```
Regnier  
Loisel
```

La méthode split retourne un tableau constitué des éléments délimités dans la ligne par le séparateur "/":

La même chose sans la variable \$mot :

```
ForEach($ligne in Get-Content c:\temp\test.txt) {($ligne.split("/"))[0]}
```

B.2.2 Écrire dans un fichier

La commande Add-Content permet d'ajouter du texte dans un fichier.

```
Add-Content -Path C:\temp\test1.txt -Value "bonjour à tous"
```

La commande Clear-Content permet de vider un fichier.

```
Clear-Content -Path C:\temp\test1.txt
```

La commande Out-File permet de rediriger le résultat d'un pipeline vers un fichier texte (unicode par défaut). Il est également possible d'utiliser les redirecteurs (> et >>).

```
ipconfig | findstr "Adresse" | Out-File c:\temp\test2.txt  
ipconfig | findstr "Adresse" > c:\temp\test2.txt
```

Export-Csv permet de générer un fichier texte avec les données séparées par une virgule et Export-Clixml permet de créer un fichier de données au format XML.

```
Get-Process | Export-Clixml c:\temp\process.xml
```

C. VARIABLES, PROPRIÉTÉS ET MÉTHODES DES OBJETS

Le nom d'une variable commence toujours par \$, il peut inclure tout caractère alphanumérique ou le trait de soulignement.

Windows PowerShell permet de créer des variables qui sont pour l'essentiel des objets nommés. La sortie de toute commande Windows PowerShell valide peut être stockée dans une variable.

Exemple

```
$loc = Get-Location
```

Il est possible d'utiliser Get-Member pour afficher des informations sur le contenu de variables.

Exemple

```
$loc | Get-Member  
Get-Location | Get-Member
```

Le nom de la variable suivi du point permet d'accéder aux propriétés de l'objet référencé par la variable, exemple pour la propriété Path de la variable \$loc.

Exemple

```
$loc.Path
```



L'usage de la touche tabulation [tab] permet de compléter le nom de la propriété.

De même, l'exécution d'une méthode (action) d'un objet.

Exemple

```
$fichier.Delete()
```



Pour les méthodes, ne pas oublier les parenthèses avec ou sans paramètre.

D. GESTION DES UTILISATEURS

D.1 LE MODULE LOCALACCOUNTS

Pour que les administrateurs puissent gérer plus facilement les utilisateurs et groupes locaux avec PowerShell, Microsoft propose une collection de cmdlets appelée « Microsoft.PowerShell.LocalAccounts ».

Le module LocalAccounts contient 15 cmdlets. Pour voir la liste complète, exécutez la commande suivante : `Get-Command -Module Microsoft.PowerShell.LocalAccounts`

• PS C:\Users\Administrateur.NS0LANFRANKFURT.002\Documents\ScriptsPS> `Get-Command -Module Microsoft.PowerShell.LocalAccounts`

CommandType	Name	Version	Source
-----	----	-----	-----
Cmdlet	Add-LocalGroupMember	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Disable-LocalUser	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Enable-LocalUser	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Get-LocalGroup	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Get-LocalGroupMember	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Get-LocalUser	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	New-LocalGroup	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	New-LocalUser	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Remove-LocalGroup	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Remove-LocalGroupMember	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Remove-LocalUser	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Rename-LocalGroup	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Rename-LocalUser	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Set-LocalGroup	1.0.0.0	Microsoft.PowerShell.LocalAccounts
Cmdlet	Set-LocalUser	1.0.0.0	Microsoft.PowerShell.LocalAccounts

D.2 LE MODULE ACTIVE DIRECTORY

Le module permettant de gérer les objets de l'Active Directory s'appelle « ActiveDirectory ». Il est théoriquement déjà installé. Pour voir la liste complète des cmdlets (très longue), exécutez la commande suivante :

`Get-Command -Module ActiveDirectory.`

Voici un extrait du résultat :

• PS C:\Users\Administrateur.NS0LANFRANKFURT.002\Documents\ScriptsPS> `Get-Command -Module ActiveDirectory`

CommandType	Name	Version	Source
-----	----	-----	-----
Cmdlet	Add-ADCentralAccessPolicyMember	1.0.1.0	ActiveDirectory
Cmdlet	Add-ADComputerServiceAccount	1.0.1.0	ActiveDirectory
Cmdlet	Add-ADDomainControllerPasswordReplicationPolicy	1.0.1.0	ActiveDirectory
Cmdlet	Add-ADFineGrainedPasswordPolicySubject	1.0.1.0	ActiveDirectory
Cmdlet	Add-ADGroupMember	1.0.1.0	ActiveDirectory
Cmdlet	Add-ADPrincipalGroupMembership	1.0.1.0	ActiveDirectory
Cmdlet	Add-ADResourcePropertyListMember	1.0.1.0	ActiveDirectory
Cmdlet	Clear-ADAccountExpiration	1.0.1.0	ActiveDirectory
Cmdlet	Clear-ADClaimTransformLink	1.0.1.0	ActiveDirectory



Une formation rapide sur ces modules est disponible ici :

<https://learn.microsoft.com/fr-fr/training/modules/manage-active-directory-domain-services-use-powershell-cmdlets/1-introduction>

E. GÉNÉRALITÉS ET BONNES PRATIQUES

E.1 LES COMMENTAIRES DE CODE

Les commentaires dans le code sont libres. Il faut néanmoins essayer de les cadrer au maximum afin de dépersonnaliser le code dans le but de le rendre maintenable par n'importe qui.

La syntaxe des commentaires

Il existe 2 types de commentaires :

1. Commentaire monoligne : le début de la ligne doit commencer par le caractère # .
2. Commentaire multilignes : le bloc de commentaires est ouvert par les caractères <# et refermé par les caractères #>.

Entête d'un fichier ps1

Il est important que dès le début d'un fichier, on sache un certain nombre d'infos : traitements réalisés, l'auteur, la date, etc. Pour cela, il faut définir un « header ». Les balises sont libres. Voici celles recommandées dans les bonnes pratiques PowerShell :

```
#=====
# NAME: XXXXXX.ps1
# AUTHOR: Nom Prénom, Entreprise
# DATE: DD/MM/YYYY
#
# VERSION 1.X
# COMMENTS: Description des traitements
#
#Requires -Version 7.4
# =====
```

Commenter une partie du code

```
<#
Description brève de la partie du code...
#>
```

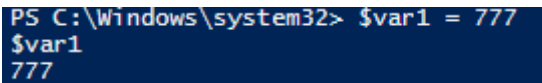
E.2 LES VARIABLES

E.2.1 Définition

Une variable est une unité de mémoire dans laquelle sont stockées des valeurs. Dans PowerShell, une variable commence par le signe dollar « \$ » et son nom peut contenir des lettres, des chiffres et des tirets.

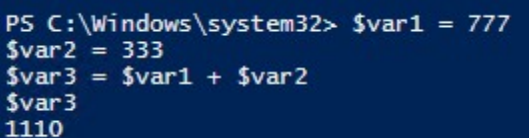
Pour attribuer une valeur à une variable, on utilise l'opérateur égale « = ». Pour afficher la valeur d'une variable, il suffit de saisir la variable. Les commandes suivantes attribuent une valeur à une variable, puis la vérifient :

```
$var1 = 777
$var1
```

A screenshot of a PowerShell terminal window. The prompt is 'PS C:\Windows\system32>'. The user enters '\$var1 = 777' and the prompt returns '\$var1'. The user then enters '\$var1' and the prompt returns '777'.

Vous pouvez effectuer des opérations arithmétiques avec des nombres :

```
$var1 = 777
$var2 = 333
$var3 = $var1 + $var2
$var3
```

A screenshot of a PowerShell terminal window. The prompt is 'PS C:\Windows\system32>'. The user enters '\$var1 = 777', '\$var2 = 333', '\$var3 = \$var1 + \$var2', and '\$var3'. The prompt returns '\$var3' and then '1110'.

Vous pouvez également combiner des chaînes de caractères à l'aide de l'opérateur plus « + » :

```
$var1 = "Complete"
$var2 = "Visibility"
$var3 = $var1 + $var2
$var3
```

```
PS C:\Windows\system32> $var1 = "Complete"
$var2 = "Visibility"
$var3 = $var1 + $var2
$var3
CompleteVisibility
```

Si vous ajoutez un nombre à une chaîne, ce nombre devient automatiquement une chaîne :

```
$var1 = "Netwrix"
$var2 = 2018
$var3 = $var1 + $var2
$var3
```

```
PS C:\Windows\system32> $var1 = "Netwrix"
$var2 = 2018
$var3 = $var1 + $var2
$var3
Netwrix2018
```

E.2.2 Types de variables

Le type de variable PowerShell est défini automatiquement d'après la valeur que vous lui attribuez, mais vous pouvez également le définir manuellement :

```
[string]$var1 = "Netwrix"
```

```
PS C:\Windows\system32> [string]$var1 = "Netwrix"
$var1.GetType().FullName
System.String
```

Pour connaître le type d'une variable, on utilise le paramètre « GetType () » :

```
$var1 = "Netwrix"
$var1.GetType().FullName
```

```
PS C:\Windows\system32> $var1 = "Netwrix"
$var1.GetType().FullName
System.String
```

Comme nous pouvons le voir, notre variable \$var1 est du type « System.String ».

Voici les descriptions de tous les types de variables PowerShell :

Type de variable	Description	Exemple
[string] System.String	Chaîne de texte	\$var1 = "Netwrix"
[bool] System.Boolean	Booléen (la valeur peut être \$true ou \$false)	\$var1 = \$true
[int] System.Int32	Nombre entier 32 bits	\$var2 = 235235345
[long] System.Int64	Nombre entier 64 bits	\$var2 = 2352353457
[decimal] System.Decimal	Nombre décimal 128 bits avec la lettre « d » à la fin	\$var2 = 23523.53456d
[double] System.Double	Nombre décimal à virgule flottante de 8 octets	[double]\$var2 = 23523.53456
[single] System.Single	Nombre décimal à virgule flottante de 32 bits	[single]\$var2 = 23523.53
[DateTime] System.DateTime	Date et heure	\$var3 = get-date
[array] System.Object[]	Tableau (décrit en détail plus loin dans l'article)	\$var3 = "premier", "deuxième", "troisième"

E.2.3 Définir une variable

Vous pouvez créer une variable simplement en lui attribuant une valeur. Par exemple, la commande « \$var4 = "exempledevariable" » crée une variable nommée **\$var4** et lui affecte une valeur de chaîne. Les guillemets doubles (") indiquent qu'une valeur de chaîne est attribuée à la variable.

E.2.4 Afficher les variables

Cette commande est très similaire à celle qui établit la liste des variables, elle utilise simplement un autre cmdlet :

```
Get-Variable | Out-String
```

E.2.5 Imprimer une variable

Vous pouvez enregistrer une variable au format .txt, .csv ou HTML. Pour l'enregistrer dans un fichier .txt, utilisez la commande **Out-File** :

```
$var5 = "Hello World!"  
$var5 | Out-File C:\scripts\Hello_World.txt
```

Pour exporter les données vers un fichier.csv, utilisez la commande **Export-Csv** :

```
$var6 = Get-Process  
$var6 | SELECT Name, Path | Export-Csv -Path C:\scripts\processes.csv
```

#TYPE Selected.System.Diagnostics.Process	
Name	Path
ALESservice	
AppVShNotify	C:\Program Files\Common Files\Microsoft Shared\ClickToRun\AppVShNotify.exe
AppVShNotify	
AuditIntelligence	
chrome	
chrome	
chrome	C:\Program Files (x86)\Google\Chrome\Application\chrome.exe
chrome	
chrome	C:\Program Files (x86)\Google\Chrome\Application\chrome.exe
chrome	C:\Program Files (x86)\Google\Chrome\Application\chrome.exe
conhost	
conhost	
csrss	

Pour lire un fichier que vous avez exporté, utilisez le cmdlet **Get-Content** :

```
Get-Content C:\scripts\processes.csv
```

```
PS C:\Windows\system32> Get-Content C:\scripts\processes.csv  
#TYPE Selected.System.Diagnostics.Process  
"Name","Path"  
"ALESservice",  
"AppVShNotify","C:\Program Files\Common Files\Microsoft Shared\ClickToRun\AppVShNotify.exe"  
"AppVShNotify",  
"AuditIntelligence",  
"chrome",  
"chrome",  
"chrome","C:\Program Files (x86)\Google\Chrome\Application\chrome.exe"  
"chrome",  
"chrome","C:\Program Files (x86)\Google\Chrome\Application\chrome.exe"  
"chrome","C:\Program Files (x86)\Google\Chrome\Application\chrome.exe"  
"conhost",  
"conhost",
```

E.2.6 Effacer une variable

```
Clear-Variable -name var6 #without $
```

E.2.7 Supprimer une variable

Pour supprimer complètement une variable, utilisez le cmdlet **Remove-Variable** :

```
Remove-Variable -name var6 #without $
```

E.2.8 Créer une variable « tableau »

Un tableau est un type de variable. C'est un ensemble de composants (les éléments du tableau) disposés dans un certain ordre. Les éléments du tableau sont numérotés de manière séquentielle, et on peut accéder à un élément en utilisant son numéro d'index.

Lorsque vous créez des tableaux, prenez en compte le comportement par défaut de PowerShell. Si vous créez un tableau avec plusieurs éléments, PowerShell créera un tableau, tel que vous le souhaitez. Placez par exemple quelques nombres dans un tableau, puis vérifiez le type de données de la variable :

```
$array1 = 1, 2, 3  
$array1.GetType()
```

Comme vous pouvez le constater, PowerShell a créé un tableau (System.Array).

E.3 LES OPÉRATEURS

E.3.1 Les opérateurs arithmétiques

Les opérateurs de comparaison permettent de comparer des données entre elles.

Voici l'ensemble des opérateurs de comparaison :

Opérateur	Signification
-eq	Égal
-ne	Non égal (différent)
-gt	Strictement supérieur
-ge	Supérieur ou égal
-lt	Strictement inférieur
-le	Inférieur ou égal

E.3.2 Les opérateurs de chaînes

Les opérateurs de chaînes vous permettent de manipuler des chaînes de caractères.

```
"Bonjour Christophe" -Replace "Christophe","Sébastien"
```

Voici la liste des opérateurs disponibles :

Opérateur	Description	Exemple
*	Répète une chaîne	"-" * 20
+	Concatène des chaînes	"Bonjour " + "Pierre"
-replace	Remplace une chaîne (Insensible à la casse)	"Bonjour Pierre" -replace "Pierre","Paul"
-creplace	Remplace une chaîne (Sensible à la casse)	"Bonjour Pierre" -replace "pierre","paul"
-eq	Vérifie l'égalité (Insensible à la casse)	"Pierre" -eq "pierre"
-ceq	Vérifie l'égalité (Sensible à la casse)	"Pierre" -ceq "pierre"
-like	Vérifie qu'une chaîne contient une autre chaîne (Insensible à la casse, Accept *)	"Pierre" -like "Pier*"
-clike	Vérifie qu'une chaîne contient une autre chaîne (Sensible à la casse, Accept *)	"Pierre" -clike "Pier*"
-notlike	Vérifie qu'une chaîne ne contient pas une autre chaîne (Insensible à la casse, Accept *)	"Pierre" -notlike "Pier*"
-cnotlike	Vérifie qu'une chaîne ne contient pas une autre chaîne (Sensible à la casse, Accept *)	"Pierre" -cnotlike "Pier*"
-match	Vérifie qu'une chaîne contient une autre chaîne (Insensible à la casse)	"Pierre" -match "rr"
-cmatch	Vérifie qu'une chaîne contient une autre chaîne (Sensible à la casse)	"Pierre" -cmatch "rr"
-notmatch	Vérifie qu'une chaîne ne contient pas une autre chaîne (Insensible à la casse)	"Pierre" -notmatch "rr"
-cnotmatch	Vérifie qu'une chaîne ne contient pas une autre chaîne (Sensible à la casse)	"Pierre" -cnotmatch "rr"

E.3.3 Les opérateurs logiques

Les opérateurs logiques permettent de vérifier plusieurs comparaisons dans la même expression.

```
$r = 5  
($r -gt 2) -and ($r -lt 8)  
True
```

Les opérateurs logiques disponibles sont

Opérateur	Signification
-and	Et logique
-or	Ou logique
-xor	Ou exclusif
-not	Non logique
!	Non logique

E.4 LES STRUCTURES DE CONTRÔLE IF, ELSEIF, ELSE, SWITCH, TRY, CATCH...

Une structure conditionnelle permet, via une évaluation de condition, d'orienter l'exécution vers un bloc d'instructions ou vers un autre. La syntaxe d'une structure conditionnelle est la suivante :

```
If (condition)
{
#bloc d'instructions
}
```

Pour mieux comprendre l'utilisation d'une structure conditionnelle, en voici quelques exemples.

Imaginons que nous souhaitons déterminer si une valeur entrée par l'utilisateur est la lettre A. Pour cela, nous allons utiliser une structure conditionnelle avec une condition sur la valeur de la variable testée. En utilisant un opérateur de comparaison, la structure est la suivante :

```
$var = Read-Host "Entrez un caractère"

If ($var -eq 'A')
{
    "Le caractère saisi par l'utilisateur est un 'A'"
}
```

Si la variable entrée par l'utilisateur est un A, alors une chaîne de caractères sera émise, sinon l'exécution poursuivra son cours sans exécuter le bloc d'instructions suivant le If.

À l'instruction If peut être associée la clause Else. Cette clause permet en cas de retour d'une valeur False d'orienter le traitement vers un second bloc d'instructions. Prenons l'exemple suivant :

```
If (($var1 -eq 15) -and ($var2 -eq 18))
{
    # Bloc d'instructions 1
}
Else
{
    # Bloc d'instructions 2
}
```

Dans un premier temps, PowerShell évalue la première condition, à savoir « la variable \$var1 est égale à 15 ». Si le test est vrai alors la première condition prend la valeur true.

Puis, il évalue la seconde condition « la variable \$var2 est égale à 18 ». Si le test est vrai alors la seconde condition prend également la valeur true.

Puis, si les deux valeurs sont vraies, l'opérateur logique -and de la condition retourne la valeur true (vrai ET vrai = vrai), et ainsi le bloc d'instruction 1 sera exécuté, sinon, si la condition est fausse, ce sera le bloc d'instructions 2 qui sera exécuté.

Toujours dans le même registre voici d'autres exemples.

Exemple 1

```
[int]$var1 = Read-Host 'Saisissez un nombre'
[int]$var2 = Read-Host 'Saisissez un nombre'
```

Exemple 2

```
If ($var1 -ge $var2)
{
    "$var1 est plus grand ou égal que $var2"
}
Else
{
    "$var1 est plus petit que $var2"
}
```

Dans ce second exemple, l'utilisateur saisit deux nombres stockés respectivement dans les variables \$var1 et \$var2. PowerShell teste ensuite si la première variable est plus grande ou égale à la seconde. Si la condition est vraie, alors nous retournons une chaîne indiquant que la première valeur est plus grande ou égale à la seconde. Si la condition est fausse, c'est la chaîne se situant dans le bloc d'instructions de la clause Else qui est retournée.

Enfin, pour finir sur les structures conditionnelles voici comment les améliorer grâce à l'instruction Elseif. L'instruction Elseif permet, si la condition précédente est fausse, de tester une autre condition. Ainsi, en utilisant une structure conditionnelle avec des Elseif, nous ne nous limitons plus à une orientation binaire, mais nous augmentons les possibles orientations du flot d'exécution.

Exemple

```
[int]$val = Read-Host 'Entrez une valeur : 1,2 ou 3'

If ($val -eq 1)
{ 'La valeur saisie est égale à 1' }
ElseIf($val -eq 2)
{ 'La valeur saisie est égale à 2' }
ElseIf($val -eq 3)
{ 'La valeur saisie est égale à 3' }
Else
{ "La valeur saisie n'est pas égale à 1 ni à 2 ni à 3 !" }
```

De cette manière, on aurait pu créer autant de Elseif que voulu. Mais l'utilisation intensive des Elseif, bien que viable, n'est pas une solution très élégante. Le fait qu'il y ait autant de conditions que de blocs d'instructions ne rend pas le code très souple ni très lisible, et on préférera s'orienter vers l'instruction Switch.

L'instruction Switch permet de remplacer avantageusement une série de « If, Elseif et Else ». À la différence de l'instruction If qui, pour une expression donnée, oriente la suite de l'exécution vers l'un des deux blocs d'instructions, l'instruction Switch oriente l'exécution vers plusieurs blocs d'instructions distincts. Et ce, avec une seule expression. Ce qui lui confère une utilisation nettement plus souple.

On peut construire un Switch de plusieurs manières, en fonction de ce que l'on veut tester. La syntaxe de Switch est la suivante :

```
Switch (<Expression>)
{
    <Valeur_1> { bloc d'instructions 1 ; Break }
    <Valeur_2> { bloc d'instructions 2 ; Break }
    <Valeur_3> { bloc d'instructions 3 ; Break }
    Default   { bloc d'instructions 4 }
}
```

La valeur Default est facultative, son bloc d'instructions n'est exécuté uniquement dans le cas où l'expression ne correspond à aucune valeur.

Il est d'usage d'utiliser l'instruction Break à l'intérieur des blocs d'instructions. Cela permet de quitter le traitement Switch en cours. En effet, si l'on exécute un bloc d'instructions, c'est que la valeur évaluée correspond à l'expression. Il n'y a donc pas lieu de tester les autres valeurs.

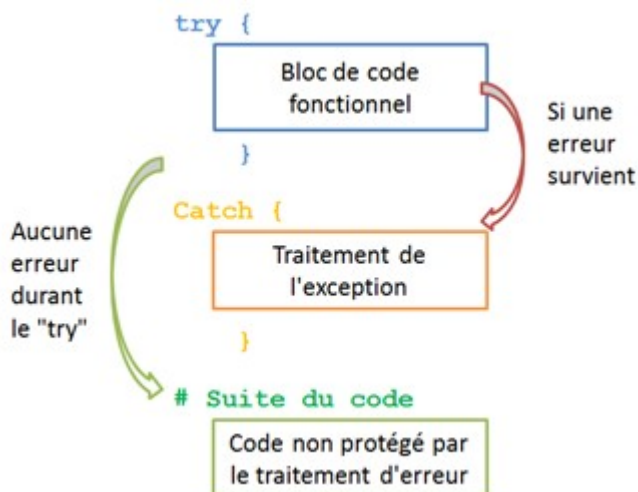
Prenons pour exemple d'application, le cas basique où l'utilisateur doit saisir un nombre entre 1 et 5, et PowerShell détermine quel nombre a été saisi. Le code est le suivant :

```
[Int]$Nombre = Read-Host 'Entrez un nombre compris entre 1 et 5 '

Switch($Nombre)
{
    1 { 'Vous avez saisi le nombre 1 ' ; Break }
    2 { 'Vous avez saisi le nombre 2 ' ; Break }
    3 { 'Vous avez saisi le nombre 3 ' ; Break }
    4 { 'Vous avez saisi le nombre 4 ' ; Break }
    5 { 'Vous avez saisi le nombre 5 ' ; Break }
    Default { "Nombre saisi incorrect !" }
}
```

Depuis Powershell v2, l'instruction trap est de plus en plus rarement utilisée au profit des blocs de code "Try" et "Catch". Cette fois-ci, on essaye un bloc de code via "Try", et en cas d'erreur on exécute les instructions contenues dans le(s) bloc(s) "Catch" suivant(s).

Le bloc "Finally" est optionnel et sera toujours exécuté.



Par exemple :

```
$array = @(3,0,1,2)
$nb = 10

foreach ($item in $array)
{
    try
    {
        $nb/$item | Out-Null
        Write-Host -Fore 'green' "Le chiffre $nb est divisible par $item"
    }
    catch
    {
        Write-Host -Fore 'red' "Erreur! Impossible de diviser $nb par $item !"
    }
    finally
    {
        Write-Host -Fore 'green' "Information - Le chiffre traité est $nb"
    }
}
```

E.5 LES BOUCLES : WHILE, FOR, FOREACH, DO.

Une boucle est une structure répétitive qui permet d'exécuter plusieurs fois les instructions se trouvant à l'intérieur du bloc d'instructions. Il est important de maîtriser le concept des boucles, car on le retrouve dans tout langage informatique et il est à la base de tout programme quel que soit le langage.

E.5.1 Boucle While

Les instructions du bloc d'instructions de cette boucle sont répétées tant que la condition de la boucle est satisfaite (est vraie). Autrement dit, dès que la condition est évaluée à la valeur \$false, on sort de la boucle.

La syntaxe d'une boucle While est la suivante :

```
While (<condition>)  
{  
    #bloc d'instructions  
}
```

Son fonctionnement est le suivant :

1. La boucle évalue la condition.
2. Si la condition est fausse, le bloc d'instruction n'est pas exécuté et la boucle se termine.
3. Si la condition est vraie, alors le bloc d'instructions est exécuté.
4. Retour à l'étape 1.

Voici un exemple basique d'une boucle While qui liste les valeurs contenues dans un tableau. Dans cette boucle, tant que la valeur \$nombre est strictement inférieure à la taille du tableau, le bloc d'instructions retourne la valeur du tableau à l'indice \$nombre.

```
$nombre = 0  
$tab = 0..99  
  
While($nombre -lt $tab.Length)  
{  
    $tab[$nombre]  
    $nombre++  
}
```

E.5.2 Boucle Do-While

La boucle Do-While s'apparente à la boucle While, à la différence près que la condition est évaluée à la fin. La boucle Do-While se structure de la façon suivante :

```
Do  
{  
    #bloc d'instructions  
}  
While (<condition>)
```

Le test de condition étant à la fin, le bloc d'instructions est exécuté au moins une fois, même si le test est faux.

E.5.3 Boucle For

La boucle For permet d'exécuter un certain nombre de fois un bloc d'instructions.

Lorsque l'on utilise une boucle For, on indique une valeur de départ, une condition de répétition de la boucle ainsi que le pas d'incrément, c'est-à-dire la valeur dont elle est augmentée à chaque itération.

La syntaxe de la boucle For est la suivante :

```
For (<initial> ;<condition> ;<incrément>)  
{  
    #bloc d'instructions  
}
```

Son fonctionnement est le suivant :

1. L'expression initiale est évaluée, il s'agit en général d'une affectation qui initialise une variable.
2. La condition de répétition est évaluée.
3. Si la condition est fausse, l'instruction For se termine.
4. Si la condition est vraie, le bloc d'instructions est exécuté.
5. L'expression est incrémentée avec le pas choisi et l'exécution reprend à l'étape 2.
- 6.

Reprenons l'exemple du parcours d'un tableau, mais cette fois-ci avec une boucle For.

```
$tab = 0..99  
For($i=0 ;$i -le 99 ;$i++)  
{  
    $tab[$i]  
}
```

E.5.4 Boucle Foreach

L'utilisation de la cmdlet Foreach-Object ou de son alias Foreach permet de parcourir les valeurs d'une collection et par chacune, exécuter une instruction.

```
$array = 1, 2, 3, 4  
foreach ($item in $array)  
{  
    Write-Host $item  
}
```